

Cravitoo · Bugs & Security Q&A Pack

Two Q&A documents to bring to your meeting when the discussion turns to bugs, vulnerabilities and security posture.

Part 1 — Bugs & Security brief YOU present to the developer

Print and hand over. Answers every question a developer needs to understand what's fixed, what's open, and where the risk currently lives.

1. Have there been any known security incidents so far?

No confirmed incidents. During development the AI-agent testing suite discovered and fixed 5 critical issues before production traffic began: - Role escalation via unauthenticated API calls - Demo/test credentials being served from production - Logout not clearing tokens (infinite loop after sign-out) - Master admin password reset-to-default on every restart - Employee session getting wiped every 15 minutes (access-token expiry with no refresh path)

Every one of these was closed by regression tests before going live.

2. What authentication model does the app use?

- **JWT** — signed with `JWT_SECRET` (HS256).
- **Access token:** 15-minute lifetime.
- **Refresh token:** 365-day lifetime, stored in mobile SecureStore (Keychain/Keystore) or `HttpOnly Secure SameSite=None` cookies on web.
- **Password hashing:** bcrypt with per-user salt.
- **OTP login:** email OTPs sent via Resend; codes hashed at rest with SHA-256; one-time use.
- **Brute-force protection:** in-memory rate limiter locks an IP+email pair after N failed logins for `LOCKOUT_MINUTES` minutes.
- **Session invalidation:** admin sets `is_active=false` on the user; the very next request returns **403 Account deactivated** on every endpoint including `/auth/refresh`.

3. What role-based access control (RBAC) exists?

Every protected endpoint calls a `get_current_user` dependency that decodes the JWT and checks the user's role against the required set. Explicit helpers: - `is_master_admin(user)` - `is_master_or_super(user)` - `can_access_site(user, site_id)` — city/site admins are scoped to

their own city/site. - Cross-tenant access is blocked at the query level (queries include `company_id` / `site_id` filters).

4. What secrets & credentials exist, and where do they live?

Secret	Where	Rotation
<code>MONGO_URL</code> , <code>DB_NAME</code>	Emergent → Manage Publishing → Secrets	Every 90 days recommended
<code>JWT_SECRET</code>	Same	Rotate on any suspected leak
<code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code>	Same (bootstrap only — real admin password lives in DB)	Every quarter
<code>RESEND_API_KEY</code>	Same	Rotate via Resend dashboard
<code>RAZORPAY_KEY_ID</code> , <code>RAZORPAY_KEY_SECRET</code>	Same	Rotate via Razorpay dashboard
<code>EMERGENT_LLM_KEY</code>	Same	Emergent-managed
No secrets in git. <code>/app/backend/.env</code> is in <code>.gitignore</code> and only holds preview values.		

5. Recently fixed bugs (already closed with regression tests)

ID	Severity	Description	Test file
S-01	Critical	Master admin password reset back to <code>admin123</code> after every backend restart	<code>tests/ test_seed_admin_password.py</code>
S-02	Critical	Access token 15 min + no refresh → employees kicked out every 15 min	<code>tests/ test_session_persistence.py</code>
S-03	Critical	<code>get_current_user</code> didn't respect <code>is_active</code> → deactivated employees could keep using the app	<code>tests/ test_session_persistence.py</code>
S-04	High	Delete Site / Vendor / Corporate Client had no cascade → orphans in <code>orders</code> , <code>menu_items</code> , <code>payment_transactions</code> , <code>vendor_onboarding</code> inflated the dashboard	curl-verified in preview
S-05	High	Demo panel (<code>/master/demo</code> + <code>routers/ demo.py</code>) shipped to production; also	routes deleted, files removed

ID	Severity	Description	Test file
		hardcoded <code>vendor@spicekitchen.com / employee@techcorp.com</code> on mobile login	
S-06	Medium	Auto <code>seed_demo_data</code> ran on every preview and staging boot	permanently disabled at startup
S-07	Medium	Mobile apps hard-coded to preview backend URL — could leak preview data to production users	<code>eas.json</code> now overrides <code>EXPO_PUBLIC_BACKEND_URL</code> per build profile

6. Known open bugs / limitations (backlog — nothing production-blocking)

- Some employee flows on mobile web (Orders, Meal Plans, Preferences) aren't yet fully responsive.
- No admin UI yet for deactivating a user — the backend endpoints exist (`POST /api/admin/users/{id}/deactivate|reactivate`), but they must be called via API for now.
- Dashboard counts update on page load, not in real time (websocket push not yet wired).
- No 2FA / TOTP for master admin login.
- No CAPTCHA on `/auth/register` — mitigated by allowed-domain whitelist.
- No rate limiting on `/auth/otp/request` beyond a 60-second cooldown per email.
- Push-token cleanup on uninstall relies on backend-marking-inactive; stale tokens may still receive one silent push before being pruned.

7. Compliance & data handling

- **PII stored:** email, name, phone, company_id, site_id. No card data ever hits Cravitoo servers — Razorpay hosts the checkout popup and returns a signed token.
- **DPDP Act (India) readiness:** user can request data export & deletion from `/settings/data` . Master admin can bulk-wipe via Reset-to-Blank for client handover.
- **Cookies:** only two — `access_token` and `refresh_token` . Both HttpOnly, Secure, SameSite=None. No third-party trackers.
- **Analytics:** none in prod (no Google Analytics / Mixpanel).
- **Retention:** audit_log rows are kept indefinitely; deleted user rows are hard-deleted (not soft).

8. Payment / financial security

- Razorpay integration uses the **standard checkout flow** — server creates order via `POST /payments/razorpay/create-order` , popup collects card, and the backend verifies the HMAC signature (`POST /payments/razorpay/verify`) before marking the order paid.
- Payment webhooks are validated via Razorpay signature header.
- Refunds are manual (Razorpay dashboard) for now.

9. Infrastructure security

- Managed by Emergent — TLS termination, DDoS at the edge, MongoDB Atlas connection with IP allowlist and TLS.
- Backend runs as a non-root process inside a Kubernetes pod.
- Automatic backups: MongoDB Atlas point-in-time (last 24h) + one weekly snapshot; Reset endpoint also creates a `_reset_backup_<timestamp>` collection before wiping.

10. What tools + tests are in place today?

- `pytest /app/backend/tests/` — regression suites for every critical security fix (S-01 through S-07 all have named tests).
- Playwright automated flows (`/app/test_reports/iteration_*.json`) — 15+ full-app smoke runs on file.
- ESLint on the frontend; ruff/basic Python lint on the backend.
- No SAST / SCA / DAST tool wired yet — this is your first ask for the developer.

11. Immediate security asks for the developer

1. Add **2FA (TOTP)** for master admin + super admin.
 2. Add **rate limiting** middleware on `/auth/otp/request` , `/auth/register` , and `/auth/refresh` .
 3. Wire **SCA** (`pip-audit` , `npm audit`) into the GitHub CI so vulnerable dependencies get flagged automatically.
 4. Add a **security.txt** file at `/.well-known/security.txt` with a responsible-disclosure contact.
 5. Consider rotating `JWT_SECRET` on production and force-re-login of all sessions.
-

Part 2 — Bugs & Security questions the developer will ask YOU

Skim before the meeting. Fill in the <...> blanks.

Access & scope

Q1. What level of access are you giving me — read-only, code-write, or full production admin? <pick one; recommend "code-write in preview + supervised production deploy" for the first month> .

Q2. Are there NDAs, security clearances or data-handling policies I need to sign? <yes/no> . If yes, will be sent in advance.

Q3. Can I use my own machine + editor, or do you require a hardened corporate laptop? <own machine – but FDE + password + auto-lock required> .

Q4. Who reviews and approves security-related PRs?
<you / a security consultant / me alone> .

Threat model

Q5. What's the worst-case breach scenario you're worried about? Real customer payment fraud through a compromised master admin OR mass leak of employee PII (emails + order history) via a leaked JWT_SECRET.

Q6. Any known active adversaries or targeted threats? No — this is a B2B corporate app in India, not high-profile enough for targeted attacks. Main risks: credential stuffing, weak admin passwords, and misconfigured deploys.

Q7. Have you engaged any pen-testers before? <yes – vendor + date / no> . Please plan for a first pass once the mobile rollout is done.

Known bugs

Q8. Give me the top 5 open bugs in priority order. 1. Mobile-web responsiveness on Orders / Meal Plans / Preferences. 2. Missing Deactivate/Reactivate UI for admins. 3. Dashboard not real-time — counts only update on refresh. 4. Vendor self-service menu editor missing (only master can edit post-approval). 5. Play Store / TestFlight rollout of the corrected production-URL mobile build not yet done.

Q9. Are there any P0/P1 bugs live in production right now? <no active P0 as of Feb 2026 – confirm on the day> .

Q10. Where do bugs get reported today? Through this Emergent chat + noted in `/app/memory/PRD.md` . No formal tracker yet — happy to move to GitHub Issues, Linear or Notion.

Q11. What's your definition of "critical" vs "high" vs "medium"? Critical = data loss, payment fraud, auth bypass, or full outage. High = any user cannot complete their core task. Medium = a workaround exists. Low = polish/UX.

Security-specific

Q12. Have you had any security audits, code reviews or vulnerability disclosures? `<yes: Emergent internal – see /app/PHASE1_FULL_AUDIT_RESULTS.md / no external audit yet>` .

Q13. Where are all secrets stored today? Emergent Manage Publishing → Secrets. Nothing in the git repo. Preview values in `/app/backend/.env` (gitignored).

Q14. What's the JWT secret rotation policy? None formally — rotate on suspected leak. Please help set up a quarterly rotation.

Q15. Do you have MFA on the Emergent, GitHub, Play Store, Apple Developer, Razorpay, Resend, and MongoDB Atlas accounts? `<yes on all – confirm in advance / no on <X>>` . Please treat this as your day-1 checklist.

Q16. What logs are captured, and for how long? FastAPI structlog to stdout → Emergent's log aggregator (~7 days). MongoDB `audit_log` collection is kept indefinitely (every admin action).

Q17. Any GDPR / DPDP Act compliance work in-flight? Yes — `/settings/data` gives a data-export + delete-account button. Master admin can bulk-wipe. No cross-border data transfer (Mongo Atlas in Mumbai region — confirm).

Q18. Are there rate limits on auth endpoints? Yes on `/auth/login` (per IP+email lockout). No formal limit on `/auth/otp/request` (only 60s cooldown per email). No limit on `/auth/register` .

Q19. Is TLS enforced end-to-end? Yes — Emergent's ingress terminates TLS 1.2+ and rewrites HTTP → HTTPS. Backend refuses `Secure=false` cookies in production (`is_secure_request` check).

Q20. What's your incident-response process today? Informal — I'm the master admin, I flip toggles / roll back via Emergent's rollback feature. Need help defining a formal runbook.

Data integrity

Q21. Any test data lingering in production? `<no – wiped via Reset-to-Blank in Feb 2026 / confirm on the day>` .

Q22. Do you take database backups? Yes — MongoDB Atlas point-in-time (24h) + weekly snapshot + `_reset_backup_<ts>` collection created before every Reset. Please add a monthly export to S3 as a belt-and-braces measure.

Q23. How do we recover from a bad deploy? Emergent "Rollback" button reverts to the previous release. Database migrations are backwards-compatible so far — no destructive schema changes shipped.

Q24. Any regulatory reporting obligations? FSSAI vendor compliance (handled at onboarding). No RBI/SEBI reporting; payments are routed through Razorpay who handles KYC.

Testing & QA

Q25. Coverage % of the codebase by tests?

<not measured yet – probably ~15-20% by lines, but every critical auth/onboarding path has a regression test> .

Q26. Do you run CI on every commit? Emergent runs lint + smoke automatically on Save-to-GitHub. Full pytest and Playwright runs are triggered manually. Please help set up GitHub Actions to run all tests on every PR.

Q27. Any bug-bounty program? No — happy to bootstrap something small (HackerOne / Bugcrowd VDP) once the developer audits our current posture.

Financial / payment

Q28. Is Razorpay in test mode or live mode? <test on preview, live on production – confirm on the day> . Live-mode key rotation available in the Razorpay dashboard.

Q29. Are refunds automated? No — manual through Razorpay dashboard. Automating this is a P2 backlog item.

Q30. What's the maximum single-transaction value? <₹? – Razorpay account default, please confirm> . Corporate orders can be higher.

After the meeting

Q31. What's the smallest useful task I can pick up first to build trust? Add the Deactivate/Reactivate button to the Admins & Employees list — backend endpoints already exist, needs only UI work. Should take under 2 hours and gives you a full round-trip through the codebase.

Q32. How do I get started? 1. Emergent workspace invite → 2. GitHub read/write → 3. Password manager share → 4. Clone repo, `yarn install` in `/app/frontend` and `pip install -r requirements.txt` in `/app/backend` → 5. Hit preview URL with master admin creds → 6. Read `/app/memory/PRD.md` in full.